

Employee Performance Monitoring + Gamification

Context

The user wants two related things: (1) a way for supervisors (Team Lead, Account Manager, Site Lead, Super Admin) to monitor their reports' performance, scoped exactly like every other visibility rule already in this app, and (2) a lightweight gamification layer (scores/badges) motivating Staff - built so it adds negligible runtime load, matching this codebase's own hard-won "one grouped query, never N+1" convention from the earlier Performance pass. This is genuinely greenfield - confirmed via a full grep sweep, nothing badge/score/leaderboard-related exists today, and the existing Reports page is still 100% mock data, not something to build on top of.

Product decisions already confirmed with the user:

- Scoring rewards volume *and* quality together - resolving more tickets helps, but a breach costs more than a resolution gains, and causing an escalation costs even more.
- **Staff** are scored on direct ticket work (attributed to whoever actually resolved it, not just whoever currently holds it). **Team Lead and Account Manager** are *also* scored - but on a different signal: how fast they personally resolve things once they've taken ownership of an escalation (the existing Escalation Handling SLA clock is a ready-made fit for this). **Site Lead/Super Admin are viewers only**, not scored, in v1.
- Monitoring scope mirrors existing visibility exactly, no new scoping model invented: Team Lead sees their own category's Staff; Account Manager sees performance tied to their own owned clients; Site Lead/Super Admin see the whole company. Everyone can always see their own scorecard regardless of role.

Why `agent\_id` can't be used for attribution

`Ticket.agent\_id` is "who's currently working it" and changes hands on claim/transfer - crediting a resolution to it would misattribute work whenever a ticket was reassigned. Instead, attribute to `AuditLog.actor\_id` on the event that actually closed the ticket - a real snapshot of who performed that specific action, immune to later reassignment.

Data model - one new table, no persisted scores

Scores are always computed live via one grouped SQL query (same reasoning as `dashboard\_stats`/`sla\_overview\_counts` already do this cheaply) - nothing to keep in sync. The only thing worth persisting is **badges earned**, since a badge is inherently "this happened, permanently record it," not a live number.

New table `performance\_badges` (new Alembic migration in `alembic\_ticketing` - this feature reads only ticketing-domain tables, and that chain already has precedent for FK'ing into `users.user\_id` without owning it, e.g. `AuditLog.actor\_id`):

```

performance_badges
  badge_id      UUID PK
  user_id       UUID, FK -> users.user_id, indexed
  badge_type    String(50)          -- plain string, same convention as notification_type
  period_start  DateTime(tz)
  period_end    DateTime(tz)
  metric_snapshot JSONB, nullable    -- e.g. {"resolved_within_sla": 22, "score": 78} for
display
  user_name     String, nullable     -- denormalized snapshot so a later-deactivated user
still displays
  awarded_at    DateTime(tz), default now()
  unique index  on (user_id, badge_type, period_end)    -- idempotency guard against
double-awarding

```

Backend: one grouped aggregation query, reusing existing scoping

New method on ``AuditLogRepository`` (or a new ``PerformanceRepository`` if that reads better - either is fine, follow whichever file already has the closest related joins), mirroring

``TicketRepository.dashboard_stats``'s exact shape: one ``select(func.count().filter(...), ...)`` with FILTER clauses, ``GROUP BY`` the attributed actor, executed via one query - never a per-row loop.

- **Period: trailing 30 days.** Long enough to smooth week-to-week noise, short enough to stay "current."
- **Staff score inputs:** resolved-within-SLA count, resolved-late count (both via ``ResolutionSLA.completed_at`` vs ``due_at``, joined off the ticket's ``STATUS_CHANGED``-to-``CLOSED`` audit-log row), and escalations-caused count.
- **Verify before implementing:** confirm the exact JSON key ``AuditLog.new_values`` actually uses for a ``STATUS_CHANGED`` event (don't assume ``"current_status"`` - check the real write site in ``interaction_service.py``'s ``change_status`` before writing the query).
- **Known limitation, worth stating plainly rather than over-engineering around:** attributing an `*auto*`-escalation (as opposed to a manual one, which already has ``triggered_by_user_id``) to a specific person is inherently approximate - attribute it to whoever was last ``actor_id`` on that ticket before the breach. Document this as an approximation, don't try to make it perfect in v1.
- **Team Lead / Account Manager score inputs:** ``TicketEscalation.acknowledged_by`` joined to ``EscalationHandlingSLA``, FILTERed on ``completed_at <= due_at`` (met) vs ``breached_at IS NOT NULL`` (breached).
- **Formula:** ``score = resolved_within_sla*2 + resolved_late*1 - escalations_caused*3`` (Staff); the equivalent handling-SLA-met/breached shape for Team Lead/Account Manager. Clamp at 0 for display, keep the raw value for sorting.
- **Scoping plug-in - reuse exactly, don't reinvent:** the same three inputs ``ticket_service.py`` already resolves - ``_resolve_category_ticket_types`` for Team Lead, the owned-clients ``account_manager_id`` pattern for Account Manager, both ``None`` for Site Lead/Super Admin (plus a ``GROUP BY`` category/client rollup for their company-wide breakdown view).

Badges (6, evaluated by a periodic job, not on every page view)

- **SLA Champion** - `>=20` resolved within SLA, 0 late, in the period.

- **Fast Responder** (Staff) - ≥ 15 First Response completions within 50% of target.
- **Zero Escalations Streak** - 0 escalations caused with ≥ 10 resolutions, two consecutive periods.
- **Escalation Resolver** (Team Lead/Account Manager) - ≥ 10 handling-SLA-met acknowledgments, 0 breaches.
- **Volume Leader** - top decile resolved-count *within their own category/client scope* (a relative rank via a window function like ``PERCENT_RANK() OVER (PARTITION BY ...)`` in the same query - not an absolute number, and not a second pass over the data).
- **Consistency Award** - $\geq 90\%$ SLA-met ratio across 3 consecutive periods.

Idempotent evaluation: the job checks ``performance_badges`` for an existing ``(user_id, badge_type, period_end)`` row before inserting; the unique index is the hard backstop against a race or double-run.

Heads up for demoing this: these thresholds (20 resolved, 15 responses, etc.) are sized for a real production volume - on the current small dev dataset, almost nobody will hit them naturally. Unlike ``SLAPolicy`` (already a live-editable table), these thresholds are plain constants in v1 - tuning them for a demo means a code change, not a settings-page edit. If you want to demo badges the way you demoed SLA breaches earlier (temporarily loosening a threshold), that's a code edit to revert afterward, not a UI toggle - flagging this now so it's not a surprise later.

API surface

New ``app/ticketing/services/performance_service.py``, new router ``app/ticketing/api/performance.py``:

- ``GET /performance/me?period_days=30`` - caller's own scorecard (any role).
- ``GET /performance/team?period_days=30`` - role-scoped list (Team Lead/Account Manager/Site Lead/Super Admin), reusing the exact scoping helpers above.
- ``GET /performance/badges/me`` - own badges.
- ``GET /performance/badges/{user_id}`` - gated by the same visibility check as ``/team``.

Response shape: ``{user_id, name, resolved_within_sla, resolved_late, escalations_caused, score, escalation_handling: {met, breached} | null}``.

Scheduler wiring

New job in ``unified-backend/app/core/sla_scheduler.py``, same singleton ``AsyncIOScheduler``, added the same way as the existing SLA sweep job: ``scheduler.add_job(_run_badge_evaluation, trigger="interval", hours=settings.badge_eval_interval_hours, id=BADGE_EVAL_JOB_ID, max_instances=1, coalesce=True, replace_existing=True)``. New ``Settings.badge_eval_interval_hours: int = 24`` (``app/core/config.py``) - daily is enough; badges are monthly-window achievements, not something needing minute-level freshness like the SLA sweep. Badge-earned notifications reuse the existing generic ``NotificationService.notify(user_ids=[user_id], notification_type="BADGE_EARNED", ...)`` - no new notification infrastructure needed.

Frontend

- ``unified-frontend/src/ticket-workspace/pages/MyPerformancePage.tsx`` - own score + badges, any role, built from ``Card`/`StatCard`` (same components the Dashboard already uses).

- ``unified-frontend/src/ticket-workspace/pages/TeamPerformancePage.tsx`` - Team Lead/Account Manager/Site Lead/Super Admin only, a ``DataTable`` of the scoped people's scores (same table component Tickets/Audit Logs already use).
- New hooks ``useMyPerformance.ts`/`useTeamPerformance.ts``, mirroring ``useDashboardSlaCounts``'s existing shape.
- New nav entries in ``NAV_ITEMS_BY_ROLE`` (``unified-frontend/src/lib/role-access.ts``) - "My Performance" for everyone, "Team Performance" gated to the four supervisor roles.

Explicitly deferred (keeps v1 reviewable, not a bigger rewrite)

- Historical trend charts over multiple past periods (v1 shows only the current trailing-30-day window).
- Admin-configurable badge thresholds (v1 is hardcoded constants, same limitation flagged above).
- Any Staff-visible peer comparison within their own team - v1 gives Staff only their own scorecard, never a ranked list of colleagues; the "Team Performance" view is supervisor-only.

Critical files

- ``unified-backend/app/ticketing/repositories/audit_log_repository.py`` (or ``ticket_repository.py``) - new aggregation method
- ``unified-backend/app/ticketing/services/performance_service.py`` - new
- ``unified-backend/app/ticketing/services/ticket_service.py`` - reuse ``_resolve_category_ticket_types``/owned-clients pattern, don't duplicate
- ``unified-backend/app/ticketing/api/performance.py`` - new router
- ``unified-backend/alembic_ticketing/versions/`` - new migration for ``performance_badges``
- ``unified-backend/app/core/sla_scheduler.py``, ``app/core/config.py`` - new job wiring
- ``unified-frontend/src/ticket-workspace/pages/MyPerformancePage.tsx``, ``TeamPerformancePage.tsx`` - new
- ``unified-frontend/src/lib/role-access.ts`` - new nav entries

Verification

- Backend: new migration applies/reverts cleanly (``alembic -c alembic_ticketing/alembic.ini upgrade head` / `downgrade -1``); confirm the aggregation query's counts match an independent raw-SQL cross-check for at least one Staff and one Team Lead account, across all three visibility scopes (Team Lead/Account Manager/Site Lead).
- Confirm the badge job is idempotent - run it twice back-to-back, confirm no duplicate rows (the unique index should reject a repeat, and the pre-check should skip it cleanly either way).
- ``npx tsc --noEmit`` clean; live-click through both new pages as Staff, Team Lead, Account Manager, and Site Lead, confirming each sees exactly the scope described above and nothing more.